

```

/* Ejemplo 10.26 p. 380 */
/* ordenar alfabeticamente una lista de cadenas de caracteres,
   utilizando una formacion de punteros */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void reordenar(int n, char *x[]);

main() {
    int i, n = 0;
    char *x[0];

    printf("Introducir debajo cada cadena en una linea\n\n");
    printf("Escribir \'FIN\' para terminar\n\n");

    /* leer la lista de cadenas de caracteres */
    do {
        /* reservar memoria */
        x[n] = (char *) malloc(12 * sizeof(char));

        printf("cadena %d: ", n + 1);
        scanf("%s", x[n]);
    }
    while (strcmp(x[n++], "FIN"));

    /* escribir la lista reordenada de cadenas de caracteres */
    printf("\n\nLista reordenada de cadenas:\n\n");
    for (i = 0; i < n; ++i)
        printf("\ncadena %d: %s", i + 1, x[i]);
}

void reordenar(int n, char *x[]) /* reordenar la lista de cadenas
{
    char *temp;
    int i, elem;

    for (elem = 0; elem < n - 1; ++elem)
        /* encontrar la menor de las cadenas restantes */
        for (i = elem + 1; i < n; ++i)

```

```

        if (strcmp(x[elem], x[i] > 0)) {
            /* intercambiar las dos cadenas */
            temp = x[elem];
            x[elem] = x[i];
            x[i] = temp;
        }
    return;
}

```

/ ejemplo 10.29 p. 389 */*

```

int procesar(int (*) (int, int)); /* prototipo de función anfitrión */
int func1(int, int); /* prototipo de función (huésped) */
int func2(int, int); /* prototipo de función (huésped) */

```

```

main()
{
    int i, j;
    i = procesar (func1) ; /* se pasa func1 a procesar; devuelve
                           valor para i */
    j = procesar (func2); /* se pasa func2 a procesar; devuelve
                           valor para j */
}

```

```

procesar(int (*pf) (int, int)) /* definición de función anfitrión
                               /* (el argumento formal es un puntero
                               a una función) */

```

```

{
    c = (*pf) (a, b); /* acceso a la función pasada a es a función
                     devuelve un valor para c */
    return(c);
}

```

```

func1(int a, int b) /* definición de función huésped */
{
    int c;
    c = /* utiliza a y b para evaluar c */
    return(c);
}

```

```

func2(int x, int y) /* definición de función huésped */

```

```

{
    int z;
    z =          /* utiliza x e y para evaluar z */
    return (z);
}

```

```

/* cálculos financieros personales */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <math.h>
/* prototipos de funciones */
void tabla(double (*pf) (double i, int m, double n), double a, int
double n),
double md1(double i, int m, double n),
double md2(double i, int m, double n),
double md3(double i, int m, double n),
main ()
{
    /* calcular el valor futuro de una serie de depósitos mensuales */
    int m, /* número de períodos de composición por año */
    double n, /* número de años */
    double a[i] /* cantidad de cada pago mensual */
    char free; /* indicador de frecuencia de composición */
    /* entrada de datos */
    printf ( "\nVALOR FUTURO DE UNA SERIE DE DEPOSITAS MENSUALES\n\n"
    printf ("Cantidad de cada pago mensual: "),
    scanf ("%lf", &a),
    printf ("Número de años: 11);
    scanf("%lf 11
    , &n)[i]
    /* introducir frecuencia de composición */
    do {
    printf ( "11 Frecuencia de composición (A1 Sr T, M, D, e): ");
    scanf ( " %ls", &frec),
    frec = toupper (frec) ;. /* convertir a mayúsculas */
    if (frec == 'A') {
    ro ;;;; 1;
    printf ( " \nComposición anual \n" ) ;
    }
    else if (frec == 'S') {
    ro= 2;

```

```

printf ( " \nComposición semestral \n" ) ,
}
else if (frec == 'T') {
ro = 4;
printf ( " \nComposición trimestral \n" ) ;
}
else if (frec == 'M') {
m = 12;
printf ( " \nComposición mensual \n" ) ,
}
394 PROGRAMACIÓN EN C
else if (frec == 'D') {
m = 360;
printf ( " \nComposición diaria \n" ) ;
}
else if (frec == 'C') {
ro = 0;
printf ( " \nComposición continua\n" ) ;
}
else
printf("\nERROR - Por favor repita\n\n");
} while (frec != 'A' && frec != 'S' && frec != 'T' &&
free 1= 'M' && free != 'D' && free != 'e/');
/* realizar los cálculos */
if (frec == 'C')
tabla (md3 , a, m, n); /* composición continua */
else if (frec 'D')
tabla (md2, a, m, n); /* composición diaria * /
else
tabla (mdl, a, m, n); /* composición anual, semestral, trimestral
o mensual */
}
void tabla (double (*pf) (double i, int m, double n), double a, int
double n)
/ * generador de la tabla (esta función acepta un puntero a otra
función como argumento)
NOTA: double (*pf) (double i, int m, double n) es un PUNTERO A
FUNCION */
{
int cont=i;
double i;
double f;
/* contador del bucle */

```

```

/* tasa de interés anual */
/* valor futuro */
}
printf (" \nTasa de interés
for (cont = 1; cont <= 20;
i ::: 0.01 * cont i
f = a * (*pf) (i, m, n);
printf(" %2d
}
return i
Cantidad futura\n\n");
++cont) {
/ * EL ACCESO A LA FUNCION
COMO UN PUNTERO */
%.2f\n", cont, f);
SE PASA
double md1(double i, int m, double n)
/ * depósitos mensuales, composición periódica * /
{
PUNTEROS 395
double factor, razon;
factor = 1 + i/m;
razon = 12 * (pow(factor, m*n) - 1) / i;
return(razon) ;
}
double md2(double i, int m, double n)
/* depósitos mensuales, composición diaria */
{
double factor, razon;
factor = 1 + i/m;
razon = (pow(factor, m*n) - 1) / (pow(factor, m/12) - 1);
return(razon) ;
}
double md3(double i, int nulo, double n)
/ * depósitos mensuales, composición continua * /
{
double razon;
razon = (exp(i*n) - 1) / (exp(i/12) - 1);
return(razon) ;
}

```